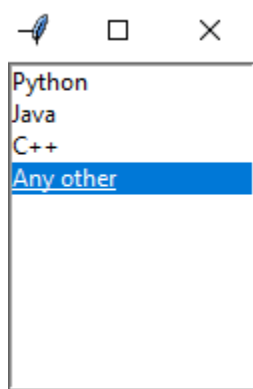


```
Lb.pack()
```

```
top.mainloop()
```



UNIT IV -----Web Development using Python

Django Installation

Installing and setting up Django is a straightforward process. Below are the step-by-step instructions to install Django and set up a new Django project on your system.

Step 1: Install Pip

Open the command prompt and enter the following command-
python -m pip install -U pip

```
C:\Windows\System32>python -m pip install -U pip
Requirement already satisfied: pip in c:\program files\python\python.exe
Collecting pip
  Using cached pip-24.0-py3-none-any.whl.metadata (3.1 kB)
Using cached pip-24.0-py3-none-any.whl (2.1 MB)
Installing collected packages: pip
  Attempting uninstall: pip
    Found existing installation: pip 23.2.1
    Uninstalling pip-23.2.1:
      Successfully uninstalled pip-23.2.1
Successfully installed pip-23.3.2
```

Step 2: Set Virtual environment

Setting up the virtual environment will allow you to edit the dependency which generally your system wouldn't allow. Follow these steps to set up a virtual environment-

Step 3: Create virtual environment in Django

We should first go the directory where we want to create the [virtual environment](#) then we type the following command to create virtual environment in django.

```
python -m venv env_site
```

```
(env_site) PS C:\Users\GFG0371\Downloads\New folder\geeks_site> python -m venv env_site
(env_site) PS C:\Users\GFG0371\Downloads\New folder\geeks_site> |
```

then we need to activate virtual environment in django

Step 4: Activate the virtual environment

Run the activation script located in the `bin` directory within the virtual environment folder

- **For Windows:**

```
.\env_site\Scripts\activate.ps1
```

Step 5: Install Django

Install django by giving following command

```
pip install django
```

```
(env_site) PS C:\Users\GFG0371\Downloads\New folder> pip install django
Collecting django
  Using cached Django-5.0.3-py3-none-any.whl.metadata (4.2 kB)
Collecting asgiref<4,>=3.7.0 (from django)
  Using cached asgiref-3.8.1-py3-none-any.whl.metadata (9.3 kB)
Collecting sqlparse>=0.3.1 (from django)
  Using cached sqlparse-0.4.4-py3-none-any.whl.metadata (4.0 kB)
Collecting tzdata (from django)
  Using cached tzdata-2024.1-py2.py3-none-any.whl.metadata (1.4 kB)
Using cached Django-5.0.3-py3-none-any.whl (8.2 MB)
Using cached asgiref-3.8.1-py3-none-any.whl (23 kB)
```

Creating Project**Django Setup**

Once Django is installed, we can start to create a new Django project.

Step 1: Start a new Django Project

Start a project by following command-

```
django-admin startproject geeks_site
```

Step 2: Navigate to the Project Directory

Change directory to geeks_site

```
cd geeks_site
```

Step 3: Start the server

Start the server by typing following command in cmd-

python manage.py runserver

```
(env_site) PS C:\Users\GFG0371\Downloads\New folder\geeks_site> python manage.py runserver
Watching for file changes with StatReloader
Performing system checks...

System check identified no issues (0 silenced).

You have 18 unapplied migration(s). Your project may not work properly until you apply the migrations.
Run 'python manage.py migrate' to apply them.
April 03, 2024 - 16:42:27
Django version 5.0.3, using settings 'geeks_site.settings'
Starting development server at http://127.0.0.1:8000/
Quit the server with CTRL-BREAK.
```

Step 4: Verify Server Status

To check whether server is running or not go to web browser and enter **http://127.0.0.1:8000/** as URL.

Creating Application in Django:-

Below is the step by step guide to create apps in django:

Step 1: Navigate to the Project Directory

Open your terminal and move into the root directory of your Django project. this is where the **manage.py** file is located.

```
cd project_name
```

Step 2: Create a New App

You can create a Django app using the following command:

```
python manage.py startapp projectApp
```

This will generate a directory structure like:

```
▼ projectName
  ▼ projectApp
    > migrations
    ● __init__.py
    ● admin.py
    ● apps.py
    ● models.py
    ● tests.py
    ● views.py
  ▼ projectName
    > __pycache__
    ● __init__.py
    ● settings.py
    ● urls.py
    ● wsgi.py
    ● manage.py
```

Step 3: Register the App in Project's settings.py

Open **project_name/settings.py** and add your app to the **INSTALLED_APPS** list:

Application definition

```
INSTALLED_APPS = [  
    'django.contrib.admin',  
    'django.contrib.auth',  
    'django.contrib.contenttypes',  
    'django.contrib.sessions',  
    'django.contrib.messages',  
    'django.contrib.staticfiles',  
    'projectApp' # Add your app here  
]
```

Step 4: Include App URLs in Project URLs

Now that we've created the app, we need to connect its URLs to the main project so that Django can properly route requests to the app's views. Here's how to do it:

Open the **urls.py** file in your main project directory (**project_name/projectApp**).

Import include from **django.urls** at the top:

from django.urls import include

In the **urlpatterns** list, add the following line to include your app's URLs:

from django.contrib import admin

from django.urls import path, include

```
urlpatterns = [  
    path('admin/', admin.site.urls), # Admin panel URL  
    path("", include("projectApp.urls")), # Include app URLs  
]
```

Step 5: Create urls.py in Your App

Your app needs its own **urls.py** to define the view routes. If it doesn't exist yet, create it inside the **projectApp** directory:

from django.urls import path

from . import views

```
urlpatterns = [  
    path("", views.index, name='index'),  
]
```

Creating View:-

Step 6: Create a View

In **projectApp/views.py**, define a view that will return a simple response:
from django.http import HttpResponse

```
def index(request):
```

```
    return HttpResponse("Hello, Geeks! Welcome to your first Django app.")
```

Optional: Update ROOT_URLCONF (Not usually recommended)

By default, Django uses the project-level `urls.py` file as the main routing file (`ROOT_URLCONF`). It's best to leave this setting as is, unless you have advanced needs.

From this:-

```
ROOT_URLCONF = 'project.urls'
```

To this:

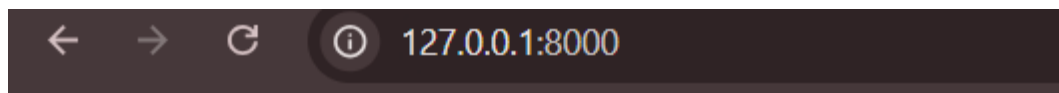
```
ROOT_URLCONF = 'app.urls'
```

Step 7: Run the Development Server

With everything set up, it's time to see your app in action. In the terminal, run:
python manage.py runserver

Then open your browser and visit:

<http://127.0.0.1:8000/>



Hello, Geeks! Welcome to your first Django app.

What is URL (Uniform Resource Locator) ?

A URL or Uniform Resource Locator is a Unique identifier that is contained by all the resources available on the internet. It can help to locate a particular resource due to its uniqueness. It is also known as the web address. A URL consists of different parts

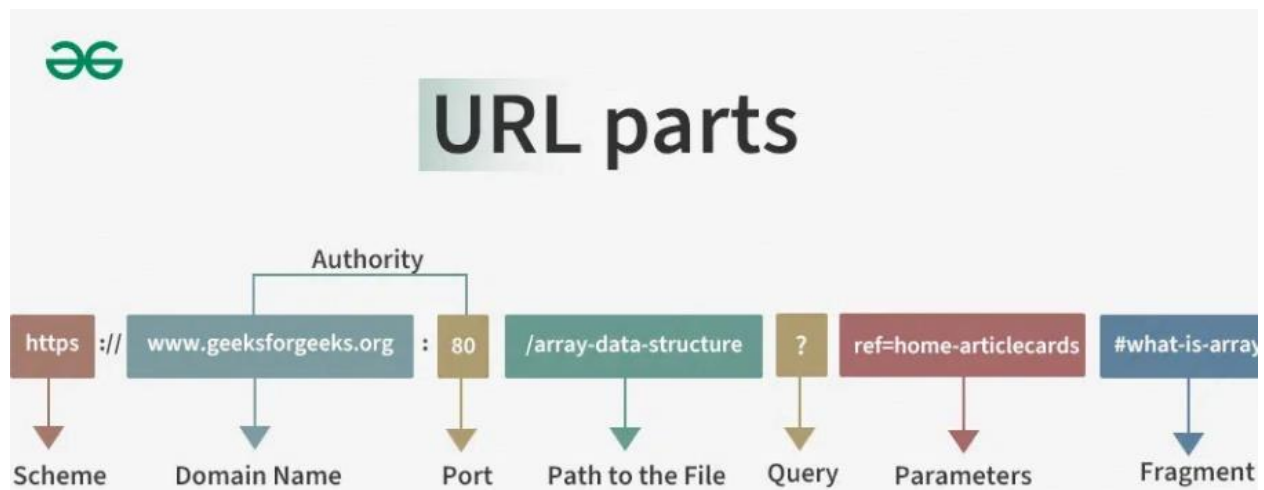
like protocol, domain name, etc. The users can access the URLs by simply typing them inside the address bar or by clicking any button or link web page.

Example URL:

`https://www.geeksforgeeks.org/`

Different Parts of a URL

A URL consists of multiple parts that can help you to visit a particular page on the internet. Every part of a URL has its own importance. Let us discuss about the different parts of a URL.



Templates

In the [Django Intro](#) page, we learned that the result should be in HTML, and it should be created in a template, so let's do that.

Create a **templates** folder inside the **members** folder, and create a HTML file named **myfirst.html**.

The file structure should be like this:

```
my_tennis_club
├── manage.py
├── my_tennis_club/
│   └── members/
│       └── templates/
│           └── myfirst.html
```

Open the HTML file and insert the following:

```
my_tennis_club/members/templates/myfirst.html:
```

```
<!DOCTYPE html>

<html>

<body>

<h1>Hello World!</h1>

<p>Welcome to my first Django project!</p>

</body>

</html>
```

Django Models

A Django model is a table in your database.

Up until now in this tutorial, output has been static data from Python or HTML templates.

Now we will see how Django allows us to work with data, without having to change or upload files in the process.

In Django, data is created in objects, called Models, and is actually tables in a database.

Create Table (Model)

To create a model, navigate to the `models.py` file in the `/members/` folder.

Open it, and add a `Member` table by creating a `Member class`, and describe the table fields in it:

```
my_tennis_club/members/models.py:
```

```
from django.db import models

class Member(models.Model):

    firstname = models.CharField(max_length=255)
```

```
lastname = models.CharField(max_length=255)
```

The first field, `firstname`, is a Text field, and will contain the first name of the members.

The second field, `lastname`, is also a Text field, with the member's last name.

Both `firstname` and `lastname` is set up to have a maximum of 255 characters.

Django Update Data

To update records that are already in the database, we first have to get the record we want to update:

```
>>> from members.models import Member
>>> x = Member.objects.all()[4]
```

will now represent the member at index 4, which is "Stale Refsnes", but to make sure, let us see if that is correct:

```
>>> x.firstname
```

This should give you this result:

```
'Stale'
```

Now we can change the values of this record:

```
>>> x.firstname = "Stalikken"
>>> x.save()
```

Execute this command to see if the Member table got updated:

```
>>> Member.objects.all().values()
```

Hopefully, the result will look like this:

```
<QuerySet [{ 'id': 1, 'firstname': 'Emil', 'lastname': 'Refsnes'},
{ 'id': 2, 'firstname': 'Tobias', 'lastname': 'Refsnes'},
{ 'id': 3, 'firstname': 'Linus', 'lastname': 'Refsnes'},
{ 'id': 4, 'firstname': 'Lene', 'lastname': 'Refsnes'},
{ 'id': 5, 'firstname': 'Stalikken', 'lastname': 'Refsnes'},
{ 'id': 6, 'firstname': 'Jane', 'lastname': 'Doe'}]>
```

Delete Records

To delete a record in a table, start by getting the record you want to delete:

```
>>> from members.models import Member  
>>> x = Member.objects.all()[5]
```

will now represent the member at index 5, which is "Jane Doe", but to make sure, let us see if that is correct:

```
>>> x.firstname
```

This should give you this result:

```
'Jane'
```

Now we can delete the record:

```
>>> x.delete()
```

The result will be:

```
(1, {'members.Member': 1})
```